

Andrés Jaramillo Barón - A01029079

Pedro Mauri Mtz - A01029143

Ricardo Calvo Pérez - A01028889

Notas de clase (Concentración de IA avanzada)

Información base del primer periodo:

- 6 semanas, en la semana 6 se trabajará el **mini reto**.

Porcentajes de evaluación:

- Tareas individuales (60%)
- Tareas en equipo (10%)
- **Exámenes:**
 - Gualberto (10%) - Lunes 8 de Septiembre
 - Esteban (10%) - Miércoles 10 de Septiembre
 - Victor (10%) - Jueves 11 de Septiembre

Políticas de la clase:

Se revisó el documento de políticas y se realizó una pequeña entrega con una firma para confirmar nuestro consentimiento de este.

<https://experiencia21.tec.mx/courses/623988/pages/politicas-del-curso>

Asistencias y faltas:

- La asistencia se tomará justo a las 5:10pm
- Debemos cumplir con el 80% de asistencia para poder tener derecho a evaluación. (máximo 5 faltas)

1 Puntualidad, asistencia y permanencia

Llegar a tiempo (dentro de los 10 primeros minutos) a todas las sesiones de clase y actividades programadas, es requerido para el buen desarrollo del curso y para considerar asistencia en la clase. *Las clases inician a los 10 minutos después de la hora, ejemplo: 7:10, 9:10, etc., y es cuando el profesor o profesora iniciará a tomar asistencia, llegar después de esos 10 minutos ya se considera falta. Las y los profesores determinarán en sus reglas de evaluación si las y los estudiantes aún con la falta, puede quedarse en la clase para no perder los contenidos, sin tener derecho a puntos en las actividades hechas ese día.*

Se espera la asistencia y permanencia de los estudiantes para no perder contenidos importantes, y por ello, **deberán cumplir con una asistencia mínima del 80% de las clases para tener derecho a la evaluación final.**

En el caso de grupos nacionales, que por su naturaleza se imparten a distancia, la plataforma de interacción se considera el espacio del salón de clases, por lo que se espera que el estudiante: tenga encendida su cámara en todo momento, responda cuando se le solicite, tenga una postura corporal propicia para el aprendizaje (tal como en el salón de clases, sentado o sentada), utilice con responsabilidad los medios de interacción.

Mini Reto:

Tendremos que crear un sistema de reconocimiento o de .

Ideas: reconocimiento de ataques de clash of clans, reconocimiento de pelea de sans, bot para farmear slayers, bot para cultivar frutas en roblox/skyblock, ia

streamer, ia para riesgos, agente para trading, ia para contar cartas, ia recetas, ia geometry dash.

Clase 1.1:

Programación tradicional:

Entrada => Reglas => Salida

a, b => a + b => R

Aprendizaje automático: (Lo que hacen las redes neuronales)

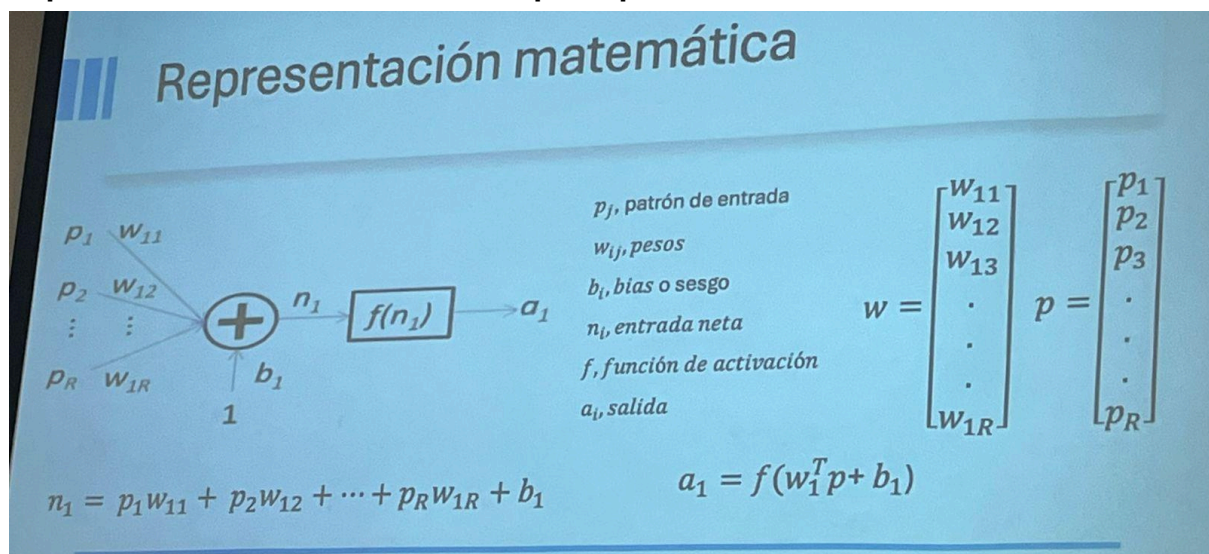
Entrada => [vacío] => Salida

Redes neuronales:

Una red neuronal es una conexión enorme de muchos **Perceptrons**.

Perceptrón similar a una **neurona**.

Representación matemática de un perceptrón:



Un uso común de las redes neuronales son los sistemas biométricos.

Sistema biométrico: Sistema para reconocer personas.

Partes de un sistema de Reconocimiento:

- Identificación (1:n)
- Verificación (1:1)

Falsos positivos VS Falsos negativos:

- **Falsos Positivos:** Si no estoy registrado, pero me deja pasar.
- **Falsos Negativos:** Si estoy registrado, pero no me deja pasar.

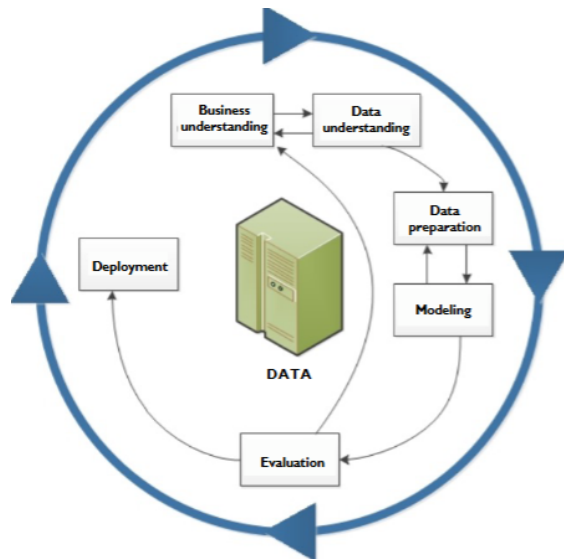
El problema es que usualmente, si uno baja, el otro sube. Por lo tanto se debe buscar un balance dependiendo de lo deseado según el contexto.

Una red neuronal tiene 3 capas de neuronas:

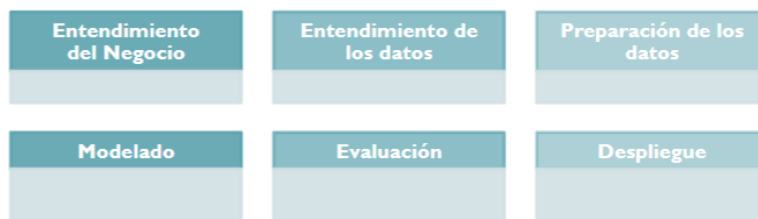
De entrada, oculta, y de salida. En la oculta puede haber un número indeterminado de neuronas, y para nosotros como personas es muy difícil el proceso que se lleva a cabo en la capa oculta por ser demasiado compleja.

Clase 1.2:

Estrategia más utilizada para la aplicación de ciencia de datos: **CRISP-DM**



Etapas:



Datos:

Tipos de datos:

- Estructurados:
 -
- No estructurados:
 -

Clase 1.3:

Todos los datos, ya sea imágenes, sonido, texto y número se les da una representación vectorial para que el algoritmo pueda entender y aprender los patrones.

Un vector para cada dato, es decir; teniendo en cuenta varias imágenes, cada una tiene su vector.

Machine Learning: Un algoritmo que identifica/detecta patrones y hace predicciones, con mayor cantidad de datos mejora su rendimiento/precisión.

Ejemplo de machine learning: Teniendo como inputs una lista de correos y una lista de etiquetas (spam o no spam), el algoritmo se encargará de organizar y determinar si es spam o no spam, y te dará el output de los correos con su respectiva etiqueta

El sistema de recomendaciones que utilizan las redes sociales para recomendarte cosas similares a las que has visto usa machine learning para realizar esas predicciones teniendo en cuenta las similitudes de la representación vectorial de los datos.

Deep learning: Rama del machine learning que utiliza redes neuronales para realizar procesos muy complicados con el objetivo de realizar tareas complejas como identificación de imágenes, reconocimiento de voz y traducción de idiomas.

El **entrenamiento/aprendizaje supervisado** va a aprender a partir de los ejemplos que nosotros les demos, como los niños en donde ellos aprenden a partir de los ejemplos que nosotros les damos, como puede ser diferenciar una manzana con una pera cuando le enseñas los dos. Tiene dos grandes fases: **fase de entrenamiento** y **fase de prueba**.

Ejemplo de entrenamiento/aprendizaje supervisado: Tengo un dataset de correos etiquetados, y este va a evaluar nuevos correos evaluando trigonometricamente a que clase de correos se acercan más.

El **entrenamiento solamente puede ser en el aprendizaje supervisado:** gold standard o ground truth 80% para entrenar y 20% para probar. Al entrenar es importante tener balanceados (vectores equitativamente divididos).

Para saber cuántos clusters hay en el no supervisado se utiliza el método de codo

El **algoritmo k-means** hace aprendizaje no supervisado, analiza vectores para agruparlos en clusters, dando posibilidad a pasar al aprendizaje supervisado; donde un experto etiqueta cada cluster.

Supervisado:

1. Al algoritmo le damos datos de dos grupos distintos (como perros y gatos), y convierte cada dato a su representación vectorial.

Clasificación binaria o multi-modal vs Regresión

Algoritmos de aprendizaje/entrenamiento supervisado:

- k-Nearest Neighbors.
- Naive Bayes.
- Support vector machines.
- Decision trees.
- Linear.
- Locally weighted linear.
- Ridge.
- Lasso.

Algoritmos de aprendizaje/entrenamiento no supervisado:

- k-Means.
- Expectation maximization.
- DBSCAN.
- Parzen window.

k-Nearest Neighbors K-NN (Algoritmo de aprendizaje supervisado): Teniendo varios datos del entrenamiento y uno nuevo de prueba, se calculan los k datos de entrenamiento más cercanos al dato de prueba (lo óptimo es que k sea impar y que sean menos de 20), y dependiendo de las etiquetas que tengan los datos k por medio del método votación (por mayoría), se le selecciona la etiqueta al nuevo dato.

Ejemplo: Si tu familia votó por el partido A, lo más seguro es que tú también lo hayas hecho al tener a tu familia como las “personas más cercanas”. Ya que el **k-Nearest Neighbors** cae en la falacia de generalización apresurada.

Clase 1.4:

Ejemplo: ¿qué función es mejor?

- Usando funciones polinomiales:

$$f(x) = c_0 + c_1x + c_2x^2 + \dots + c_kx^k$$

- ¿cuál grado es el mejor?:

- $k=0$ (constante)
- $k=1$ (recta)
- $k=2$ (parábola), etc.

- El grado no puede pasar el número de puntos.

i	Tamaño	Costo
1	500	1000
2	1300	2000
3	2000	3000
4	3800	4500
5	4000	5000
6	5500	5100
7	6800	5300
8	7400	5400
9	10000	6000

El error es la distancia entre un punto real y un punto dado por la recta, este error se calcula de la siguiente manera (esto solo consigue el error en **UN SOLO PUNTO**):

$$E = \hat{y} - y_{real}$$

Para minimizar el error se debe de considerar todas las observaciones al mismo tiempo, esto se logra de la siguiente manera:

$$E_T = \sum_{i=1}^m |\hat{y}_i - y_i| = \sum_{i=1}^m |\theta_0 + \theta_1 x_i - y_i| \quad \text{(Pero esto no se usa xd)}$$

en donde: $\hat{y}_i$ = salida obtenida, y_i = salida deseada, m = los puntos de la recta

$$\hat{y}_i = \theta_0 + \theta_1 x_i$$

Para resolver la pregunta **¿Cuánto deben de valer θ_0 y θ_1 (parámetros) para que el E_T sea mínimo?**

$$E_T = \sum_{i=1}^m (\hat{y}_i - y_i)^2 \rightarrow$$

$$1. \quad \frac{\delta}{\delta \theta_0} = \sum_{i=1}^m (\hat{y}_i - y_i)^2$$

$$1.1. \quad \frac{\delta}{\delta \theta_0} = \sum_{i=1}^m (\theta_0 + \theta_1 x_i - y_i)^2 = \sum_{i=1}^m 2(\theta_0 + \theta_1 x_i - y_i) \rightarrow \sum_{i=1}^m (\theta_0 + \theta_1 x_i - y_i) = 0$$

$$1.2. \quad m\theta_0 + \theta_1 \sum_{i=1}^m x_i - \sum_{i=1}^m y_i = 0 \rightarrow m\theta_0 + \theta_1 \sum_{i=1}^m x_i = \sum_{i=1}^m y_i$$

$$2. \quad \frac{\delta E_T}{\delta \theta_1} = \frac{\delta}{\delta \theta_1} \sum_{i=1}^m (\hat{y}_i - y_i)^2$$

$$2.1. \quad = \frac{\delta}{\delta \theta_1} \sum_{i=1}^m (\theta_0 + \theta_1 x_i - y_i)^2 = \sum_{i=1}^m \frac{\delta}{\delta \theta_1} (\theta_0 + \theta_1 x_i - y_i)^2 = \sum_{i=1}^m 2(\theta_0 + \theta_1 x_i - y_i) x_i = 0$$

$$2.2. \quad \sum_{i=1}^m \theta_0 x_i + \sum_{i=1}^m \theta_1 x_i^2 - \sum_{i=1}^m x_i y_i = 0 \rightarrow \sum_{i=1}^m \theta_0 x_i + \sum_{i=1}^m \theta_1 x_i^2 = \sum_{i=1}^m x_i y_i$$

$$2.3. \quad \theta_0 \sum_{i=1}^m x_i + \theta_1 \sum_{i=1}^m x_i^2 = \sum_{i=1}^m x_i y_i$$

3. Resultados

$$3.1. \quad m\theta_0 + \theta_1 \sum_{i=1}^m x_i = \sum_{i=1}^m y_i$$

$$3.2. \quad \theta_0 \sum_{i=1}^m x_i + \theta_1 \sum_{i=1}^m x_i^2 = \sum_{i=1}^m x_i y_i$$

Función de costo:

$$E(\theta_0, \theta_1) = \sum_{i=1}^m (\theta_0 + \theta_1 x_i - d_i)^2 \text{ (función de costo)}$$

Primer método para ajustar función de predicción a los datos:
(Regresión lineal)

Segundo método:
(Método Normal)

Handwritten notes on a piece of paper showing the matrix formulation of the normal equations for linear regression:

- Equation: $\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3$
- Design matrix X (m x 4): $X = \begin{bmatrix} 1 & x_1 & x_2 & x_3 \\ \vdots & \vdots & \vdots & \vdots \end{bmatrix}$
- Target vector y (m x 1): $y = \begin{bmatrix} y_1 \\ \vdots \end{bmatrix}$
- Parameter vector θ (4 x 1): $\theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \\ \theta_3 \end{bmatrix}$
- Normal equations: $X^T X \theta = X^T y$
- Final model (circled): $\theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3$

Facilitando el manejo de matrices

- Si se le agrega a cada ejemplo un 1 al inicio:

$$X = \begin{bmatrix} 1 & x^{(1)} \\ \dots & \dots \\ 1 & x^{(m)} \end{bmatrix} \text{ y } \theta = \begin{bmatrix} \theta_0 \\ \theta_1 \end{bmatrix}$$

- Entonces podemos escribir: $h_{\theta}(X) = X\theta$ o $\theta^T X^T$ es decir, La función de costo es: $J = (X\theta - y)^2$
- Si derivamos, igualamos a 0 y despejamos θ , directamente obtenemos la **ecuación normal** para θ :
$$\theta = (X^T X)^{-1} X^T y$$
- Sin embargo, **no se utiliza** este método por el **cálculo de la inversa**.
- El que se usa es un método iterativo llamado **Gradiente Descendente**.

Tercer Método:

Algoritmo Final

- Repetir hasta convergencia:
 - $\theta_0 = \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})$
 - $\theta_1 = \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})x^{(i)}$
- Hay variantes del algoritmo dependiendo de cómo se toman los ejemplos:
 - Batch: Cada paso de gradiente descendente usa TODOS los ejemplos de entrenamiento.
 - On-Line: Cada paso toma un ejemplo.
 - Semi-Batch: Cada paso toma s ejemplos.

$$\theta_0 = \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^m (\theta_0 + \theta_1 x_i - y_i)$$

$$\theta_1 = \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^m (\theta_0 + \theta_1 x_i - y_i) \underline{x_i}$$

Hay que multiplicar

Clase 1.5:

Paso 1 transformar a vector

Paso 2 entrenar y generar modelo

Paso 3 se crea una nueva muestra

Paso 4 predecir

Es mejor usar una tupla para almacenar los vectores de entrenamiento

cd y ad extraen y transforman datos pero los ad utilizan la herramienta que mejor se adapte, mientras que cd crea su propia herramienta.

cd pueden obtener datos que no son accionables para toma de decisiones, mientras que ad obtienen datos accionables para toma de decisiones.

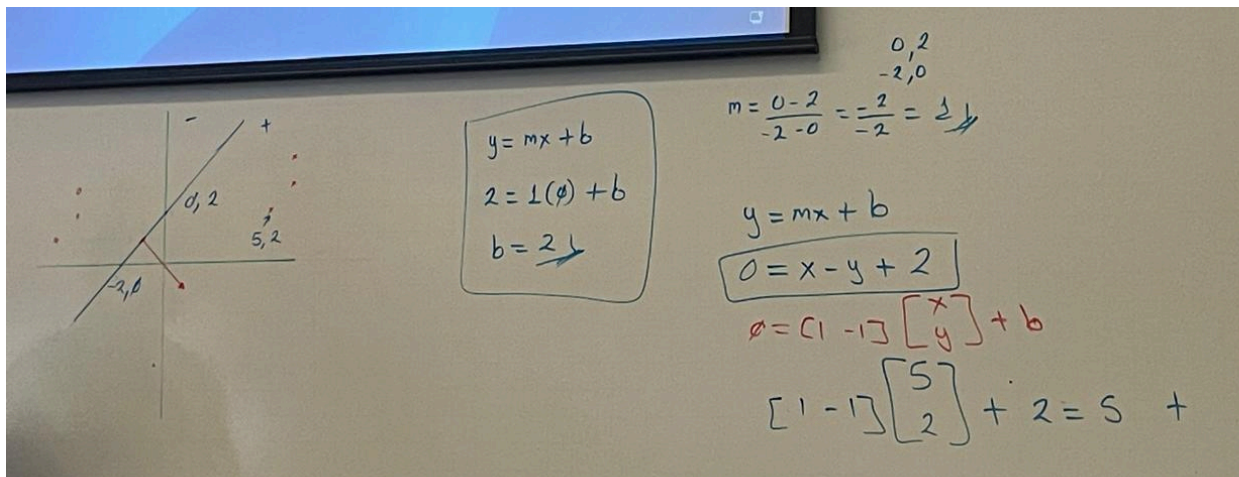
cd se necesita el conocimiento científico a profundidad sobre el dato, mientras que en ad simplemente conocimiento básico y utilizar el dato para toma de decisiones.

Clase 2.1:

Repaso de perceptrón simple.

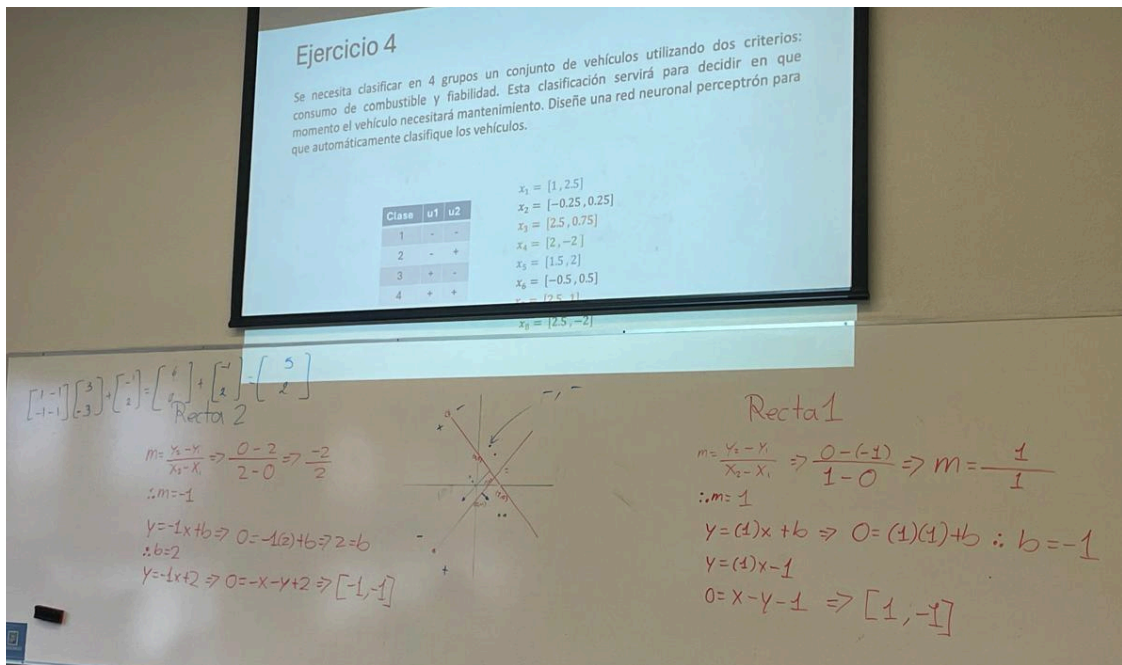
El perceptrón tiene la labor de aprender a separar muestras por clases, el simple sólo puede identificar dos clases.

Ejercicio de perceptrón simple usando una recta:



Introducción a perceptrón de 4 clases.

Ejercicio:



Introducción de ¿Cómo entrenar un perceptrón simple?

Pasos:

¿Cómo entrenar un perceptrón simple?

- Asignar valores aleatorios a los pesos.
- Realizar la sumatoria respectiva para encontrar el valor de "y".
- A la salida de "y" se le debe aplicar la función de activación
- Calcular el error respectivo.
- Si el error es igual a cero pasamos al siguiente patrón
- Si el error es diferente de cero recalculamos pesos.

$W_0 = 0.5$
 $W_1 = 1.2$
 $W_2 = 2.3$

$$y = W_0 X_0 + W_1 X_1 + W_2 X_2$$
$$y = 0.5(1) + (1.2)(1) + (2.3)(1) = 4 \rightarrow f(4) = 1$$
$$e = 5 - y = 1 - 1 = 0$$
$$y = W_0 X_0 + W_1 X_1 + W_2 X_2$$
$$y = 0.5(1) + 1.2(1) + 2.3(-1) = -0.6 \rightarrow f(-0.6) = -1$$
$$e = -1 - 1 = -2$$

b	X ₁	X ₂	y
1	1	1	1
1	1	-1	-1
1	-1	1	-1
1	-1	-1	-1

Fórmulas:

Salida:

$$y = f\left(\sum_{i=1}^n W_i X_i\right) + (\theta * W_0)$$
$$f(x) = \begin{cases} 1 & \text{si } x \geq 0 \\ -1 & \text{si } x < 0 \end{cases}$$

Error:

$e = \text{salida deseada} - \text{salida obtenida (y)}$

Calcular pesos:

$$W_i = W_i + (\eta * e * X_i)$$

Ejercicio 1:

Salida:

$$y = f\left(\sum_{i=1}^n W_i X_i\right) + (\theta * W_0)$$

$$f(x) = \begin{cases} 1 & \text{si } x \geq 0 \\ -1 & \text{si } x < 0 \end{cases}$$

Error:

$e = \text{salida deseada} - \text{salida obtenida } (y)$

Calcular pesos:

$$W_i = W_i + (\eta * e * X_i)$$

Handwritten notes on the left:

$W_0 = 1.5$
 $W_1 = 1.2$
 $W_2 = 2.3$
 $\eta = 0.5$

$y = W_0 X_0 + W_1 X_1 + W_2 X_2$
 $y = 1.5(1) + 1.2(1) + 2.3(1) = 2.6 \rightarrow f(2.6) = 1$
 $e = d - y = 1 - 1 = 0$

$y = W_0 X_0 + W_1 X_1 + W_2 X_2$
 $y = 1.5(1) + 1.2(1) + 2.3(-1) = -2.6 \rightarrow f(-2.6) = -1$
 $e = 1 - (-1) = 2$

$y = W_0 X_0 + W_1 X_1 + W_2 X_2$
 $y = 1.5(1) + 1.2(1) + 2.3(1) = 2.6 \rightarrow f(2.6) = 1$
 $e = 1 - 1 = 0$

Handwritten calculations in the center:

$y = -1.5(1) + 1.2(-1) + 2.3(-1) = -5 \rightarrow f(-5) = -1$
 $e = -1 - (-1) = 0$

$W_0 X_0 + W_1 X_1 + W_2 X_2$
 $\begin{bmatrix} 1.2 & 2.3 \end{bmatrix} \begin{bmatrix} X_1 \\ X_2 \end{bmatrix} = 1.5$
 $1.2X + 2.3Y = 1.5$
 $X = \frac{-2.3Y + 1.5}{1.2} = \frac{1.5}{1.2} - \frac{2.3}{1.2}Y$
 $Y = \frac{-1.2X + 1.5}{2.3} = \frac{2.7}{2.3} - \frac{1.2}{2.3}X$

Handwritten table on the right:

X_0	X_1	X_2	y
1	1	1	1
1	1	-1	-1
1	-1	1	-1
1	-1	-1	-1

Handwritten graph on the right:

Ejercicio 2:

Handwritten table on the left:

X_0	X_1	X_2	X_3	y
1	1	1	1	1
1	1	-1	1	1
1	-1	-1	1	1
1	1	1	-1	1
1	-1	1	-1	1
1	1	-1	-1	1
1	-1	-1	-1	1

Handwritten notes on the left:

$W_0 = 0.5$
 $W_1 = 0.5$
 $W_2 = 0.2$
 $W_3 = 0.8$
 $\eta = 0.5$

Handwritten calculations on the right:

$W_0 = 2$
 $W_1 = 0.8$
 $W_2 = 0.7$
 $W_3 = 0.2$

$[0.8, 0.7, 0.2] \begin{bmatrix} X_1 \\ X_2 \\ X_3 \end{bmatrix} + 2$

$y = X_0 W_0 + X_1 W_1 + \dots = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} + W_0 =$

Handwritten graph on the right:

Clase 2.2:

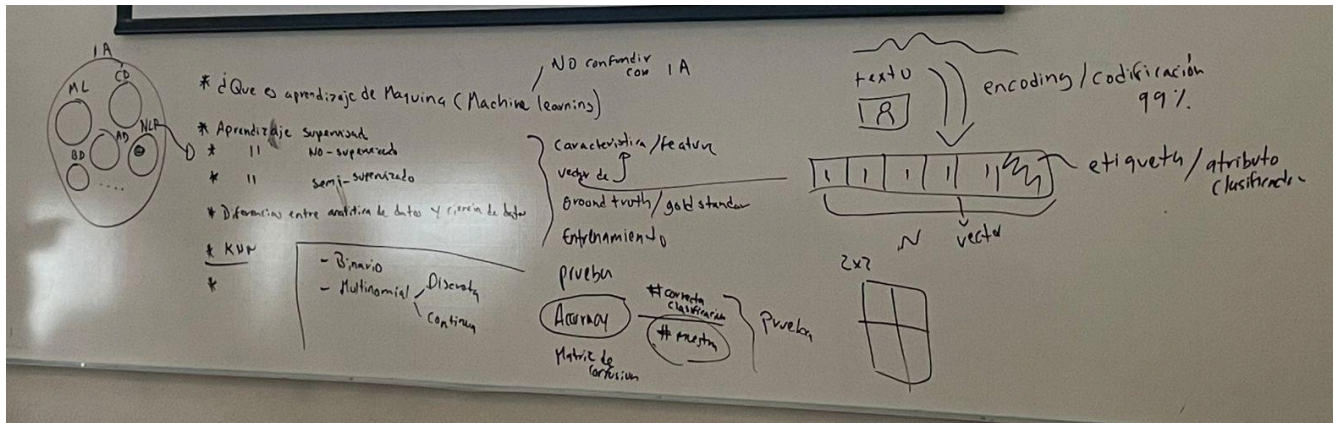
- Hiper Parámetros: Definidos por el usuario x épocas ¿cuándo me detengo?
- Parámetros -> definidos por el método q0 y q1

Algoritmo Final

- Repetir hasta convergencia:
 - $\theta_0 = \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})$
 - $\theta_1 = \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})x^{(i)}$
- Hay variantes del algoritmo dependiendo de cómo se toman los ejemplos:
 - Batch: Cada paso de gradiente descendente usa TODOS los ejemplos de entrenamiento.
 - On-Line: Cada paso toma un ejemplo.
 - Semi-Batch: Cada paso toma s ejemplos.

- Si tienes bajo error en training y en testing, es un buen modelo.
- Si tienes bajo error en training pero alto en testing, es un mal modelo. (overfitting)
- Si tienes un alto error en training, mejor no pruebes el testing, es muy mal modelo.

Clase 2.3:



Clase 3.1:

Redes conectadas a sí mismas.

RED DE HOPFIELD:

Fórmula para calcular matriz de pesos:

RED DE HOPFIELD

• MATRIZ DE PESOS

$$W_{ij} = \begin{cases} \frac{1}{N} \sum_{s=0}^{M-1} C_i^s C_j^s, & i \neq j \\ 0 & i = j \end{cases}$$

N = NÚMERO DE NEURONAS (tamaño de las C, todas tienen el mismo tamaño)

S = NÚMERO DE PATRONES (Número de C (patrones) que se evalúan)

C = PATRONES QUE ESTAMOS EVALUANDO

I = POSICIÓN 1 DENTRO DE LOS PATRONES (i, j)

J = POSICIÓN 2 DENTRO DE LOS PATRONES (i, j)

Ejercicio 1 de matriz de pesos:

$C^0 = [0, 1, 0]$
 $C^1 = [1, 1, 0]$

$W_{ij} = \begin{bmatrix} 0 & 1/3 & 0 \\ 1/3 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$
3x3

$W_{0,1} = W_{1,0} = \frac{1}{3} ((0*1) + (1*1)) = \frac{1}{3}$
 $i=0, j=1$

$W_{0,2} = W_{2,0} = \frac{1}{3} ((0*0) + (1*0)) = 0$
 $i=0, j=2$

$W_{1,2} = W_{2,1} = \frac{1}{3} ((1*0) + (1*0)) = 0$
 $i=1, j=2$

Ejercicio 2 de matriz de pesos:

$C^1 = [1, 1, 0, 1, 1]$
 $C^2 = [0, 0, 1, 0, 0]$

$W_{ij} = \frac{1}{5} \begin{bmatrix} 0 & \frac{1}{5} & 0 & \frac{1}{5} & \frac{1}{5} \\ \frac{1}{5} & 0 & 0 & \frac{1}{5} & \frac{1}{5} \\ 0 & 0 & 0 & 0 & 0 \\ \frac{1}{5} & \frac{1}{5} & 0 & 0 & \frac{1}{5} \\ \frac{1}{5} & \frac{1}{5} & 0 & \frac{1}{5} & 0 \end{bmatrix}$

$W_{0,1} = \frac{1}{5} ((1 \times 1) + (0 \times 0)) = \frac{1}{5}$
 $W_{0,2} = \frac{1}{5} ((1 \times 0) + (0 \times 1)) = 0$
 $W_{0,3} = \frac{1}{5} ((1 \times 1) + (0 \times 0)) = \frac{1}{5}$
 $W_{0,4} = \frac{1}{5} ((1 \times 1) + (0 \times 0)) = \frac{1}{5}$
 $W_{1,2} = \frac{1}{5} ((1 \times 0) + (0 \times 1)) = 0$

$W_{1,3} = \frac{1}{5} ((1 \times 1) + (0 \times 0)) = \frac{1}{5}$
 $W_{1,4} = \frac{1}{5} ((1 \times 1) + (0 \times 0)) = \frac{1}{5}$
 $W_{2,3} = \frac{1}{5} ((0 \times 1) + (1 \times 0)) = 0$
 $W_{2,4} = \frac{1}{5} ((0 \times 1) + (1 \times 0)) = 0$
 $W_{3,4} = \frac{1}{5} ((1 \times 1) + (0 \times 0)) = \frac{1}{5}$

Capacidad de la Red:

Fórmula recomendable para calcular el número de patrones a recordar:

$$0.13 * \text{Neuronas} = \text{patrones}$$

Pasos para calcular una Red de Hopfield:

Pasos para una Red de Hopfield

1. Crear la matriz de pesos
2. Aplicar la función de activación al vector de entrada
3. Obtener $U_{(n+1)} = W * y_n^T$
4. Obtener $y_{n+1} = f(U_{(n+1)})$
5. Si $y_{n+1} = y_n$, la red convergió,
6. Si $y_{n+1} \neq y_n$, regresar al paso 3

1. Crear matriz de pesos
2. Aplicar la función de activación al vector de entrada.
 - a. Si $x \geq 0$: return 1
 - b. Si $x < 0$: return -1
3. Obtener $U(n+1) = W * Y_n$.
4. Obtener $Y(n+1) = f(U(n+1))$.
 - a. Tras realizar el paso 3, aplicar función de activación en la salida, y comparar con la entrada (salida = entrada??)
5. Si $Y(n+1) = Y_n$, la red convergió. (La salida y entrada son iguales)
6. Si $Y(n+1) \neq Y_n$, regresar al paso 3 usando la salida distinta como la nueva entrada en el paso 3.

CLASE 3.2

Pregunta de examen:

CUANDO LOS DATOS SON DISCRETOS Y SE QUIERE USAR GRADIENTE ASCENDENTE NO ES POSIBLE, SE UTILIZA HILL CLIMBING PORQUE AL SER DATOS DISCRETOS NO SON DERIVABLES

CLASE 3.3

Naive Bayes:

Algoritmo de detección de palabras dentro de un vocabulario.

Lo que se hace es lo siguiente:

Se lee un texto completo, en el cual todas las palabras usadas en este se guardan en un diccionario, con el número de veces que se repite cada palabra. Este diccionario es el **vocabulario** que el modelo usará.

Entrenamiento:

Le pasa muchos vectores al algoritmo y los implementa al modelo.

Pruebas:

Le pasa vector a vector al modelo, para que el modelo prediga o pronostique.

CLASE 3.4

Pregunta de examen:

Porque no se pudo resolver el perceptrón de multi capa: Porque no se tiene un resultado deseado en los pesos de en medio.

Algoritmos de aprendizaje (redes neuronales):

Dos algoritmos de aprendizaje principales, son:

- Perceptrón
- Adaline

El algoritmo es prácticamente el mismo, con la distinción que Adaline no utiliza la función de activación ya que esta no es derivable.

Backpropagation: Algoritmo de aprendizaje Multi-capa:

lol

CLASE 3.5

Temas/algoritmos de machine learning vistos (con Esteban): **KNN, Regresión logística, Naive Bayes.**

- **¿Que tienen en común?**
 - Todos estan el scikit learn
 - Todos son algoritmos supervisados
 - Son multinomiales (se pueden adaptar para más de una clase)
- **¿Para que se usa?**
 - **KNN:**
 - Funciona con conjuntos pequeños.
 - Funciona con datos no correlacionados ni normalizados.
 - Solo te dice la clase a la que es más probable a la que pertenezca.
 - **Regresión logística:**
 - Funciona bien con datos grandes.
 - Funciona con datos normalizados.
 - Solo te dice la clase a la que es más probable a la que pertenezca.
 - **Naive Bayes:**
 - Funciona bien con datos normalizados y sin normalizar.
 - Te dice la clase a la que es más probable a la que pertenezca e información adicional del porcentaje de las otras clases.
- **¿Cuándo es mejor usarlo?**
 - **KNN:**
 - Cuan Los ejemplos son muy pequeños, casi microscópicos.
 - **Regresión logística:**
 - Asumiendo que hay una interdependencia/correlación entre los datos.
 - **Naive Bayes:**
 - “La vieja confiable”. Cuando se asume que los datos son independientes.

ej.

llegan datos de un sensor de temperaturas de un dia las etiquetas son el estado del clima, la mejor opción es la regresión logística teniendo en cuenta que los distintos sensores (humedad, oxígeno, etc) están correlacionados.

Es un **sistema supervisado** si tenemos **etiquetas** entre nuestros datos

CLASE 4.1

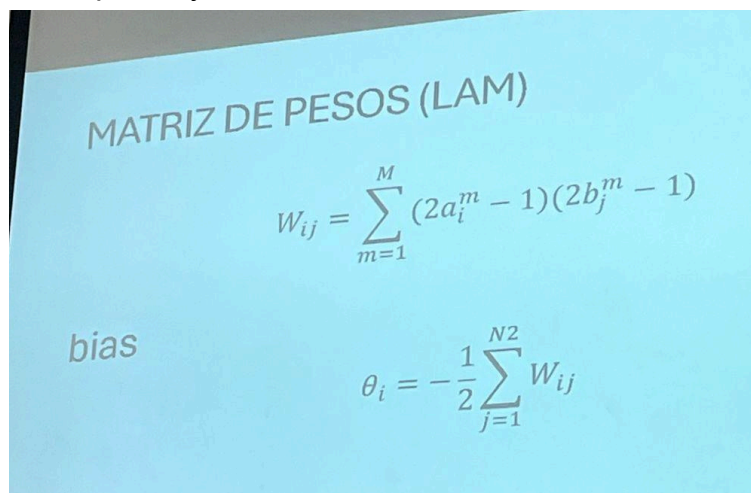
Tarea final de Gualberto: Evaluar la red hopfield y redes de la clase (asociativas) y hacer un reporte IEE. El tema general es abierto. (se puede hacer en dúos)

Redes asociativas:

- Red de Hopfield (Clase pasada)
- LAM - Memoria Asociativa Lineal. (Clase de hoy)
- BAM - Memoria Asociativa Bidireccional (Clase de hoy después del break)

LAM:

Fórmula de matriz de pesos y bias:

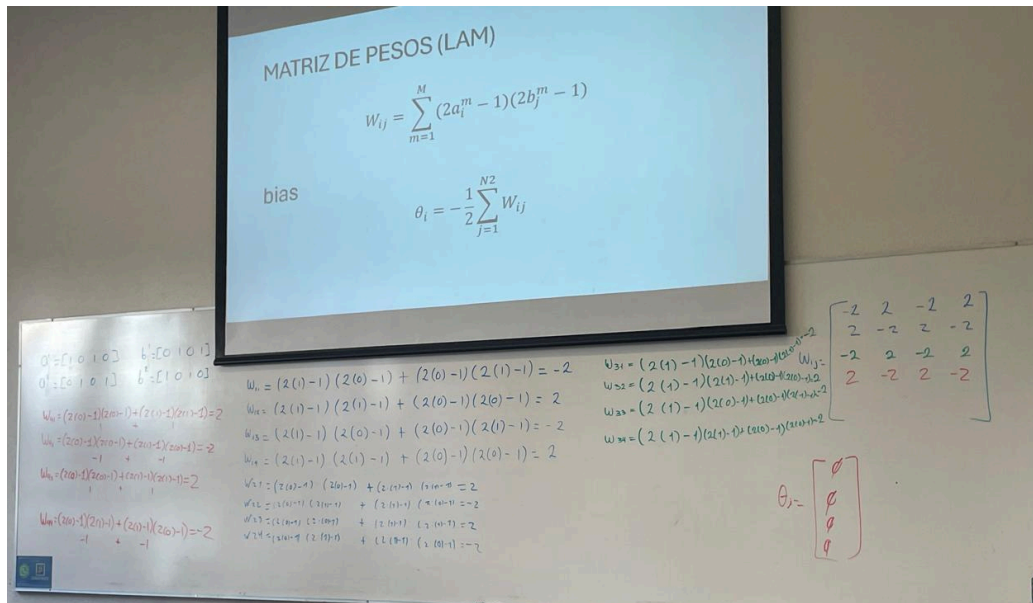


The slide displays the formulas for the LAM (Linear Associative Memory) matrix weights and bias. The title is "MATRIZ DE PESOS (LAM)". The weight formula is
$$W_{ij} = \sum_{m=1}^M (2a_i^m - 1)(2b_j^m - 1)$$
 and the bias formula is
$$\theta_i = -\frac{1}{2} \sum_{j=1}^{N2} W_{ij}$$
. The word "bias" is written to the left of the second formula.

W_{ij} = matriz de pesos, 'a' siendo la entrada, 'b' siendo la salida

Se calcula la fórmula de M_{ij} en 'dos ciclos for', repitiendo el proceso b_j veces para cada a_i veces.

Ejercicio de M_{ij} y bias:



Pruebas de la matriz de pesos:

Para calcular la salida y comprobar que la matriz sea correcta:

$$y = f(W_{ij} * a + \theta)$$

Entrada y salida deseada:

$$[1 \ 0 \ 1 \ 0] \Rightarrow [0 \ 1 \ 0 \ 1]$$

Handwritten calculations on a whiteboard:

$a^1 = [1 \ 0 \ 1 \ 0]$ $b^1 = [0 \ 1 \ 0 \ 1]$
 $a^2 = [0 \ 1 \ 0 \ 1]$ $b^2 = [1 \ 0 \ 1 \ 0]$

$y = f(W * a^1 + \theta)$

$y = \begin{bmatrix} -2 & 2 & -2 & 2 \\ 2 & -2 & 2 & -2 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} -4 \\ 4 \\ -4 \\ 4 \end{bmatrix} \Rightarrow \begin{bmatrix} 0 \\ 1 \\ 0 \\ 1 \end{bmatrix}$

Al ingresar la entrada en la fórmula, la salida fue igual a la salida esperada, por lo tanto la matriz es correcta.

Para multiplicar un vector por una matriz:

$$\begin{matrix}
 P0 \\
 P1 \\
 P2 \\
 P3
 \end{matrix}
 \begin{pmatrix}
 1 & 2 & 3 & 4 \\
 2 & 4 & 6 & 8 \\
 3 & 6 & 9 & 12 \\
 4 & 8 & 12 & 16
 \end{pmatrix}
 \times
 \begin{pmatrix}
 1 \\
 2 \\
 3 \\
 4
 \end{pmatrix}
 =
 \begin{pmatrix}
 1 \times 1 + 2 \times 2 + 3 \times 3 + 4 \times 4 \\
 2 \times 1 + 4 \times 2 + 6 \times 3 + 8 \times 4 \\
 3 \times 1 + 6 \times 2 + 9 \times 3 + 12 \times 4 \\
 4 \times 1 + 8 \times 2 + 12 \times 3 + 16 \times 4
 \end{pmatrix}$$

BAM (bidireccional):

Fórmula de matriz de pesos:

MATRIZ DE PESOS (BAM)

$$W_{ij} = \sum_{m=1}^M x_i^m y_j^m$$

Ejercicio de matriz de pesos:

$X^1 = [-1 \ 1 \ -1]$ $y^1 = [1 \ -1 \ 1]$
 $X^2 = [-1 \ -1 \ -1]$ $y^2 = [-1 \ -1 \ -1]$
 $X^3 = [1 \ 1 \ 1]$ $y^3 = [1 \ 1 \ 1]$

$w_{1,1} = x_1^1 y_1^1 + x_1^2 y_1^2 + x_1^3 y_1^3 = (-1)(1) + (-1)(-1) + (1)(1) = 1$ $w_{2,1} = (1)(1) + (-1)(-1) + (1)(1) = 3$ $w_{3,1} = (-1)(1) + 1 + 1 = 1$
 $w_{1,2} = (-1)(-1) + (-1)(-1) + (1)(1) = 3$ $w_{2,2} = (1)(-1) + 1 + 1 = 1$ $w_{3,2} = (-1)(-1) + 1 + 1 = 3$
 $w_{1,3} = (-1)(1) + (-1)(1) + (1)(1) = 1$ $w_{2,3} = (1)(1) + 1 + 1 = 3$ $w_{3,3} = (-1)(1) + 1 + 1 = 1$

$$\begin{bmatrix}
 1 & 3 & 1 \\
 3 & 1 & 3 \\
 1 & 3 & 1
 \end{bmatrix}
 \begin{bmatrix}
 1 \\
 -1 \\
 1
 \end{bmatrix}
 =
 \begin{bmatrix}
 -1 \\
 5 \\
 -1
 \end{bmatrix}$$

GUIA EXAMEN:

XOR no se puede resolver con perceptrón lineal porque no es linealmente divisible

Tabla comparativa de desempeño			
Accuracy en Training	Accuracy en Testing	Interpretación	Caso típico
Bajo	Bajo	El modelo no aprendió lo suficiente ni siquiera en los datos de entrenamiento.	Underfitting
Alto	Bajo	El modelo memorizó demasiado los datos de entrenamiento, pero no generaliza bien.	Overfitting
Medio/Alto	Medio/Alto (similar a training)	El modelo aprendió patrones reales y generaliza bien a nuevos datos.	Modelo bien ajustado
Bajo/Medio	Más alto que training	Puede pasar si el set de testing es más fácil o si hubo algún problema de muestreo.	Inusual, revisar datos